

Microsoft Fabric Master Data Engineer

6-Month Exhaustive Training & Certification Program

Advanced Architecture, Hands-on Labs, and MLOps

Abstract

This 24-week program is designed to transform you from a practitioner to a Master Data Engineer on Microsoft Fabric. It progressively builds from OneLake foundations to advanced Medallion architectures, integrating daily theory, weekly service-specific mini-projects, architectural use case analyses, and massive monthly Capstone projects. A tracking timetable is included at the end.

Contents

1 Milestone 1: Core Data Infrastructure & OneLake (Month 1)	2
2 Milestone 2: Data Warehousing & BI Integration (Month 2)	6
3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)	11
4 Milestone 4: Real-Time Intelligence (Month 4)	17
5 Milestone 5: Data Science & AI Operationalization (Month 5)	21
6 Milestone 6: Security, Administration & Exam Prep (Month 6)	26
7 Appendix A: Glossary	31
8 Appendix B: Assessment & Grading Model	32
9 Appendix C: Interview Preparation	32
10 Appendix D: Self-Check Answer Key	33
11 Appendix E: Datasets & Project Data	37
12 Appendix F: Portfolio & Presentation Guide	37
13 6-Month Progress Tracking Timetable	39

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** Working knowledge of Python (PySpark preferred), T-SQL fundamentals, Git basics, command-line comfort, and general cloud concepts. No prior Microsoft Fabric experience is required; Azure portal familiarity helps.
- **Weekly cadence:** Plan for 10–12 hours per week. Monday–Wednesday cover theory and concepts, Thursday is a mandatory hands-on project, and Friday is an architecture use case review.
- **Deliverables:** Every monthly capstone must produce a Git repository, an architecture diagram, a capacity cost estimate, and a security review checklist, as defined in the Acceptance Criteria boxes.
- **Assessment:** The 24-week tracking timetable at the end of this document doubles as the assessment checklist — a week is only “done” when its project is committed and reproducible.

Environment setup checklist:

- ☐ Install Python 3.x, Git, Docker Desktop, and VS Code (with Python and Power BI/Fabric extensions).
- ☐ Install the Azure CLI (az) and sign in; install SQL Server Management Studio (SSMS) for the SQL analytics endpoint.
- ☐ Create a free Microsoft Fabric trial account (60-day trial capacity) and enable Fabric in your tenant.
- ☐ Clone your course repository structure: `data/`, `src/`, `tests/`, `infra/`, `docs/`.
- ☐ Configure credentials via environment variables (e.g., `AZURE_CLIENT_ID`, connection strings in Azure Key Vault) — never hardcode secrets in notebooks or pipelines.

1 Milestone 1: Core Data Infrastructure & OneLake (Month 1)

Objective: Master OneLake, Workspaces, Lakehouses, and the Medallion Architecture.

Learning outcomes: provision workspaces and Lakehouses, virtualize external data with OneLake shortcuts, transform data with Spark notebooks, optimize Delta tables with V-Order and time travel

Week 1: OneLake & Fabric Foundations

Outcomes: provision a Fabric workspace and Lakehouse, virtualize external data with OneLake shortcuts, explain capacities and domains

- **Monday:** Introduction to Microsoft Fabric: SaaS architecture, Capacities, and Domains.
- **Tuesday:** OneLake concepts: The “OneDrive for Data”, Workspaces, and Folders.
- **Wednesday:** OneLake Shortcuts: Virtualizing data from ADLS Gen2, Amazon S3, and Data-verse.
- **Thursday:** **Hands-on Project:** Create a Fabric Workspace. Provision a Lakehouse and create a OneLake shortcut to a public Azure Blob Storage container without moving any data.

- **Friday: Use Case & Architecture:** Architect a multi-cloud data virtualization strategy for a global retailer that has historical sales in AWS S3 and current CRM data in Dataverse.

Resources: Microsoft OneLake documentation (<https://learn.microsoft.com/fabric/onelake/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).

Self-check:

- What does a OneLake shortcut do instead of copying data?
- What is the difference between a capacity and a workspace?
- Name two external sources supported by OneLake shortcuts.
- Why is Fabric described as SaaS rather than PaaS?
- What problem do domains solve in a large Fabric tenant?

Week 2: The Medallion Architecture

Outcomes: apply Bronze/Silver/Gold layering, choose Lakehouse vs. Warehouse per workload, isolate medallion layers across workspaces

- **Monday:** Medallion principles: Bronze (Raw), Silver (Validated), and Gold (Enriched).
- **Tuesday:** Lakehouse vs. Data Warehouse in Fabric: When to choose which.
- **Wednesday:** Organizing workspaces and Lakehouses for Medallion scaling.
- **Thursday: Hands-on Project:** Deploy three Lakehouses (Bronze, Silver, Gold). Manually upload raw CSV files to Bronze, and set up the folder structure for future transformations.
- **Friday: Use Case & Architecture:** Design a multi-workspace architecture for a financial institution ensuring strict isolation between raw ingested logs and curated financial reports.

Resources: Implement medallion lakehouse architecture in Fabric (<https://learn.microsoft.com/fabric/onelake/onelake-medallion-lakehouse-architecture>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- Which data quality level belongs in Bronze vs. Silver?
- When would you choose a Warehouse over a Lakehouse?
- Why separate workspaces per medallion layer?
- Which Delta component underlies every Lakehouse table?
- Who should hold write access to Gold, and why?

Week 3: Apache Spark in Fabric

Outcomes: configure Spark starter and custom pools, cleanse CSV data into Delta with PySpark, combine Spark SQL and PySpark in one notebook

- **Monday:** Spark compute in Fabric: Starter pools vs. Custom pools, Spark environments.
- **Tuesday:** Fabric Notebooks: PySpark basics, reading/writing files in OneLake.
- **Wednesday:** Spark SQL and combining PySpark with SQL in the same notebook.

- **Thursday: Hands-on Project:** Write a PySpark notebook that reads raw CSV data from the Bronze Lakehouse, drops duplicates, drops nulls, and writes to the Silver Lakehouse as Delta tables.

```
# Medallion cell: Bronze (CSV) -> Silver (Delta) with dedup
from pyspark.sql import functions as F

bronze_df = (spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("Files/bronze/raw_sales"))

silver_df = (bronze_df
    .dropDuplicates(["order_id"])          # dedup on business key
    .dropna(subset=["order_id", "order_date"])
    .withColumn("ingest_ts", F.current_timestamp()))

(silver_df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("silver.sales_clean"))
```

- **Friday: Use Case & Architecture:** Design a scalable ingestion layer for a telemetry system processing 100 GB of JSON logs daily using Spark Notebooks.

Resources: Apache Spark in Fabric docs (<https://learn.microsoft.com/fabric/data-engineering/spark-compute>); Apache Spark repository (<https://github.com/apache/spark>). **Self-check:**

- What is the difference between a starter pool and a custom pool?
- What does `dropDuplicates(["order_id"])` guarantee?
- Why write Silver as Delta instead of plain Parquet files?
- When would you mix Spark SQL and PySpark in one notebook?
- What is the risk of `inferSchema` on a 100 GB CSV load?

Week 4: Delta Lake Optimization

Outcomes: optimize Delta tables with V-Order, OPTIMIZE, and Z-Ordering, recover deleted data via time travel, set a safe VACUUM retention

- **Monday:** Delta Lake internals: Parquet files + Transaction logs (JSON).
- **Tuesday:** Optimizing Delta Tables: V-Order, OPTIMIZE, and Z-Ordering.
- **Wednesday:** Managing Delta Tables: VACUUM, Time Travel, and Schema Evolution.
- **Thursday: Hands-on Project:** Run an OPTIMIZE command with V-Order on a large Silver table. Then run a DR restore drill: deliberately delete or overwrite a Gold table, restore it via Delta time travel (RESTORE or a timestamp query), and verify row counts match the pre-delete baseline — this exercises the recovery path described in the Production Note below.

```
# DR drill: restore an overwritten/deleted Gold table via Delta time travel
before = spark.read.table("gold.sales_summary").count()    # baseline

spark.sql("DELETE FROM gold.sales_summary WHERE sale_date < '2024-01-01'")
print("after delete:", spark.read.table("gold.sales_summary").count())
```

```
# Option 1: restore the table to a previous version
spark.sql("RESTORE TABLE gold.sales_summary TO VERSION AS OF 3")

# Option 2: inspect a point-in-time snapshot without restoring
# spark.sql("SELECT COUNT(*) FROM gold.sales_summary "
#           "TIMESTAMP AS OF '2024-06-01T00:00:00Z'").show()

restored = spark.read.table("gold.sales_summary").count()
assert restored == before, f"DRILL FAIL: {restored} != baseline {before}"
print("restore verified:", restored)
```

- **Friday: Use Case & Architecture:** Troubleshoot and re-architect a severely degraded Delta table that has accumulated millions of small files (small file problem) over 6 months of streaming.

Production Note: Delta time travel is your first line of accidental-delete recovery in Fabric: OneLake has no traditional backup/restore, so a bad overwrite or dropped partition is recovered with RESTORE or VERSION AS OF against Delta history. Pair it with soft-delete patterns (tombstone flags) for row-level mistakes, and watch your VACUUM retention — vacuuming below the retention horizon destroys the very history you rely on.

Re-

sources: Use Delta tables in Apache Spark in Fabric (<https://learn.microsoft.com/fabric/data-engineering/lakehouse-and-delta-tables>); Delta Lake repository (<https://github.com/delta-io/delta>). **Self-check:**

- Which two components make up a Delta table?
- What does V-Order do to the underlying Parquet files?
- How does OPTIMIZE address the small-file problem?
- Why can an aggressive VACUUM break your time-travel recovery plan?
- Which command restores a table to a specific version?

CAPSTONE PROJECT - Milestone 1: Build a Spark-driven Medallion Lakehouse. Create Workspaces for Bronze, Silver, and Gold. Use OneLake shortcuts to ingest sample data. Write parameterized Spark notebooks to clean the data (Silver) and aggregate it into a Star Schema (Gold) using Delta format with V-Order enabled.

Acceptan

- End-to
- Archite
the thr
- At leas
demon
- Monito
showin
- Month
mente
- Securit
noted.

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture:** OneLake shortcuts → Bronze Lakehouse (raw files) → parameterized Spark notebooks → Silver (validated Delta) → Gold star schema; chosen for zero-copy multi-cloud ingestion and engine-agnostic Delta storage.
- Create three Lakehouses (Bronze/Silver/Gold) and shortcut external data into Bronze — no copy jobs.
- Parameterize notebooks by source path and batch date; run Bronze → Silver → Gold in order from a pipeline.
- Write Silver with dedup on the business key, explicit schemas, and Delta schema enforcement.
- Build Gold as fact + conformed dimensions; enable V-Order and OPTIMIZE with Z-ORDER on join keys.
- Capture Monitoring Hub runs and Delta history for the time-travel recovery demo.

```
# Gold optimization: compact files and co-locate join keys
spark.sql("OPTIMIZE gold.fact_sales ZORDER BY (customer_key)")
spark.sql("OPTIMIZE gold.dim_customer ZORDER BY (customer_key)")
# Verify compaction took effect
(spark.sql("DESCRIBE DETAIL gold.fact_sales")
  .select("numFiles", "sizeInBytes").show())
```

- **Pitfall:** inferSchema on large CSVs scans everything and mis-infers — declare explicit schemas.
- **Pitfall:** small-file buildup from frequent appends; schedule OPTIMIZE, do not run it ad hoc.
- **Pitfall:** spark.read.table only sees the attached Lakehouse by default — use the SQL endpoint or shortcuts for cross-Lakehouse reads.

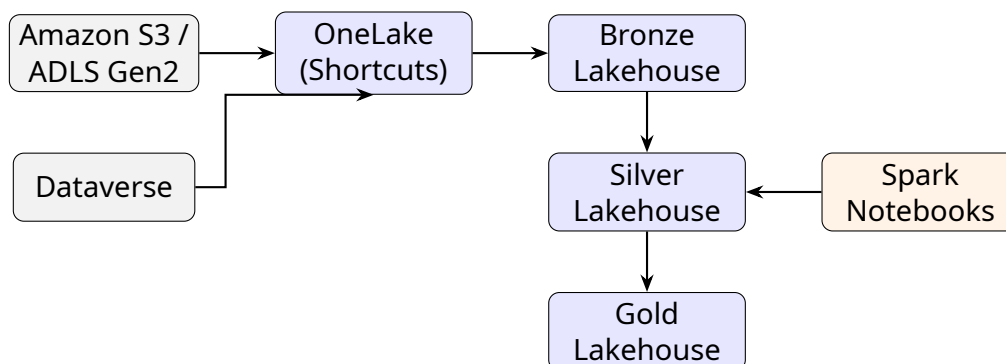


Figure 1: Milestone 1 reference architecture: OneLake medallion lakehouse

2 Milestone 2: Data Warehousing & BI Integration (Month 2)

Objective: Deep dive into Synapse Data Warehouse, T-SQL, and DirectLake mode.

Learning outcomes: load warehouses with COPY INTO, model star schemas with semantic models and DAX, secure data with RLS, build DirectLake Power BI reports

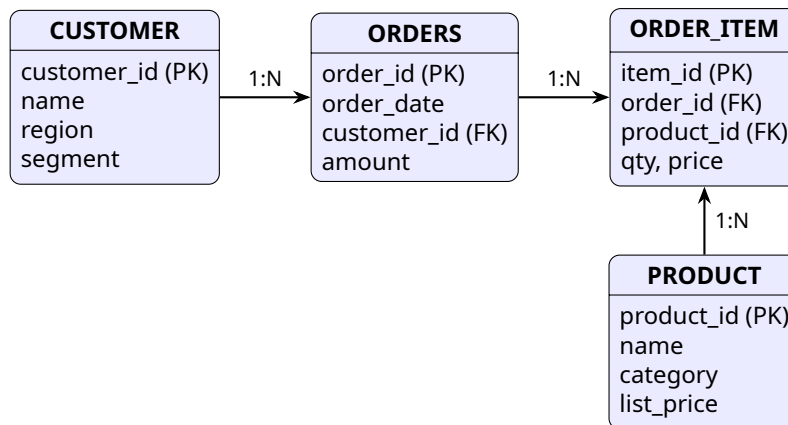


Figure 2: Capstone data model: medallion sales star (N:M orders-products via ORDER_ITEM)

Week 5: Synapse Data Warehouse in Fabric

Outcomes: load millions of rows with COPY INTO, compare pipeline/T-SQL/Dataflow ingestion patterns, estimate capacity cost for a warehouse workload

- **Monday:** Fabric Data Warehouse architecture: Compute and Storage separation.
- **Tuesday:** T-SQL in Fabric: Supported surface area, COPY INTO, and cross-database querying.
- **Wednesday:** Data Warehouse ingestion patterns (Pipelines vs. T-SQL vs. Dataflows).
- **Thursday: Hands-on Project:** Provision a Data Warehouse. Use the COPY INTO statement via a SQL Query to load 5 million rows of Parquet data directly from a OneLake path.
- **Friday: Use Case & Architecture:** Estimate capacity usage and recommend an ingestion pattern (Lakehouse vs. Warehouse) for a legacy SQL Server migration requiring heavy stored procedure usage.

Capacity & cost trade-offs (supports Friday's use case):

Option	Billing model	Best for	Watch-outs
Fabric capacity reservation (F2-F64, CU-based)	Reserved CUs, monthly/annual commit	Steady production workloads with predictable load	Paying for idle CUs off-hours; throttling if under-sized
Fabric pay-as-you-go (F2-F64)	Per-second CU billing; pause/resume any time	Dev/test, bursty or business-hours-only workloads	Easy to forget to pause; un-optimized Spark jobs spike cost
Power BI Premium (P SKU / Premium Per User)	Per-capacity or per-user monthly	BI-heavy estates already on Premium licensing	P SKUs being superseded by F SKUs; PPU limits sharing to PPU users
Fabric free trial	60-day trial capacity, no cost	Learning, proofs of concept, this syllabus	Expires; unsuitable for production SLAs

Production Note: Make COPY INTO loads idempotent: stage files under date-partitioned OneLake paths, drive loads from a manifest or watermark table, and design retries so re-running the same batch never produces duplicate rows. On failure, rerun the batch — do not hand-patch partial tables.

Re-

sources: Fabric Data Warehouse documentation (<https://learn.microsoft.com/fabric/data-warehouse/>)
Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).

Self-check:

- When is COPY INTO preferable to a pipeline Copy activity?
- Why does separating compute from storage matter for warehouse scaling?
- Which ingestion pattern best fits a stored-procedure-heavy migration?
- What is the unit of Fabric capacity billing?
- Why pause a pay-as-you-go capacity outside business hours?

Week 6: SQL Analytics Endpoint

Outcomes: query Lakehouse tables via the SQL analytics endpoint, build reusable views and TVFs, enforce object-level and row-level security

- **Monday:** The automatic SQL Analytics Endpoint for Lakehouses (Read-only).
- **Tuesday:** Creating Views, TVFs, and Stored Procedures over Lakehouse Delta tables.
- **Wednesday:** Security: Object-level and Row-level security (RLS) in the SQL Endpoint.
- **Thursday: Hands-on Project:** Connect to a Lakehouse's SQL Analytics Endpoint via SQL Server Management Studio (SSMS). Create a View joining a Silver Delta table with a Gold Delta table.
- **Friday: Use Case & Architecture:** Design a logical data warehouse architecture allowing data analysts to query raw data lake files using purely T-SQL without duplicating the data.

Resources: SQL analytics endpoint documentation (<https://learn.microsoft.com/fabric/data-engineering/lakehouse-sql-analytics-endpoint>); DuckDB repository (<https://github.com/duckdb/duckdb>).

Self-check:

- Why is the SQL analytics endpoint read-only?
- Which object would you create to reuse a join across reports?
- How does RLS differ from object-level security?
- What happens when you query a shortcut through the endpoint?
- Why connect via SSMS instead of only the web editor?

Week 7: Data Modeling for Power BI

Outcomes: model a Kimball star schema with declared grain, implement SCD Type 2 merges, define relationships and DAX measures in a semantic model

- **Monday:** Kimball dimensional modeling: declare the grain first (what one fact row means), fact table types (transaction, periodic snapshot, accumulating snapshot), conformed dimensions shared across fact tables, and Slowly Changing Dimensions — Type 1 (overwrite) vs. Type 2 (versioned history with effective dates).
- **Tuesday:** The default Power BI semantic model in Fabric Workspaces.
- **Wednesday:** Creating custom Semantic Models and defining relationships.

- **Thursday: Hands-on Project:** Open the Semantic Model view in a Fabric Workspace. Define a 1-to-many relationship between a Date dimension and a Sales fact table. Create a DAX measure. Then implement the SCD Type 2 MERGE below against your Gold Lakehouse and verify that a changed customer attribute closes the old row and opens a new one.

```
# SCD Type 2 upsert into gold.dim_customer via Delta MERGE (PySpark)
from delta.tables import DeltaTable

tgt = DeltaTable.forName(spark, "gold.dim_customer")
src = spark.read.table("silver.customer_daily") # today's snapshot

# Step 1: expire current rows whose tracked attributes changed
(tgt.alias("t").merge(src.alias("s"),
    "t.customer_id = s.customer_id AND t.is_current = true")
    .whenMatchedUpdate(
        condition="t.segment <> s.segment OR t.region <> s.region",
        set={"is_current": "false", "valid_to": "s.change_date"})
    .execute())

# Step 2: insert new versions (changed attributes + brand-new customers)
(tgt.alias("t").merge(src.alias("s"),
    "t.customer_id = s.customer_id AND t.is_current = true"
    " AND t.segment = s.segment AND t.region = s.region")
    .whenNotMatchedInsert(values={
        "customer_id": "s.customer_id", "segment": "s.segment",
        "region": "s.region", "is_current": "true",
        "valid_from": "s.change_date", "valid_to": "null"})
    .execute())
```

- **Friday: Use Case & Architecture:** Resolve a circular dependency issue in a complex enterprise data model by implementing a proper bridge table architecture.

Resources: Power BI semantic models documentation (<https://learn.microsoft.com/power-bi/connect-data/service-datasets-understand>); dbt Core repository (<https://github.com/dbt-labs/dbt-core>). **Self-check:**

- Why must you declare the grain before adding fact columns?
- Name the three Kimball fact table types.
- What makes a dimension “conformed”?
- When is SCD Type 2 required instead of Type 1?
- In the MERGE snippet, what closes the current SCD2 row?

Week 8: Power BI DirectLake Mode

Outcomes: build a DirectLake report, detect and diagnose DirectQuery fallback, benchmark DirectLake against DirectQuery

- **Monday:** DirectLake concept: Reading Delta/Parquet directly into memory (No Import, No DirectQuery).
- **Tuesday:** DirectLake fallback behavior (falling back to DirectQuery).
- **Wednesday:** Optimizing models for DirectLake (V-Order importance).
- **Thursday: Hands-on Project:** Create a Power BI report using DirectLake mode against your Gold Lakehouse. Benchmark the performance vs a traditional DirectQuery model.

- **Friday: Use Case & Architecture:** Architect a real-time BI dashboard for an executive team that requires sub-second response times on 100-million-row datasets without nightly refresh delays.

Production Note: *DirectLake silently falls back to DirectQuery when a query exceeds framing guardrails or hits unsupported shapes. Monitor fallback events with the Capacity Metrics app and semantic model telemetry — a slow “DirectLake” report is often a fallback problem or missing V-Order, not a capacity problem.*

Re-

sources: DirectLake overview (<https://learn.microsoft.com/fabric/get-started/direct-lake-overview>)
Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- What triggers a DirectLake fallback to DirectQuery?
- Why does V-Order matter specifically for DirectLake?
- What does framing do in a DirectLake model?
- Which telemetry reveals fallback events in production?
- When is Import mode still preferable to DirectLake?

CAPSTONE PROJECT - Milestone 2: End-to-End Analytics Workflow. Establish a Synapse Data Warehouse. Create complex T-SQL stored procedures to build a Star Schema from raw OneLake data. Build a custom Semantic Model with relationships and DAX measures, and publish a DirectLake Power BI report.

Acceptance

- End-to-end model
- Architecture Semantic
- At least with a
- Monitor events
- Monthly column
- Security

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture:** OneLake Delta staging → Warehouse via COPY INTO → T-SQL star schema → custom semantic model (DAX, RLS) → DirectLake Power BI; chosen for T-SQL-heavy transformation with sub-second BI.
- Stage Parquet under date-partitioned OneLake paths driven by a manifest/watermark table.
- COPY INTO warehouse staging tables; stored procedures build dimensions and facts (SCD2 where history matters).
- Create a custom semantic model: relationships, measures, and RLS roles tested as multiple test users.

- Benchmark DirectLake vs. DirectQuery and capture fallback events in the Capacity Metrics app.

```
-- Idempotent load: only files not yet in the manifest
COPY INTO dbo.stg_sales
FROM 'https://onelake.dfs.fabric.microsoft.com/<ws>/<lh>/Files/stage/sales/2024-06/'
WITH ( FILE_TYPE = 'PARQUET' );

INSERT cfg.load_manifest (batch_path, rows_loaded, loaded_ts)
VALUES ('2024-06', @@ROWCOUNT, CURRENT_TIMESTAMP);
```

- **Pitfall:** re-running COPY INTO without a manifest duplicates rows — make loads idempotent.
- **Pitfall:** DirectLake silently falls back to DirectQuery without V-Order; verify with telemetry, not assumption.
- **Pitfall:** RLS “works” only for the developer role unless you test View-As for every role.

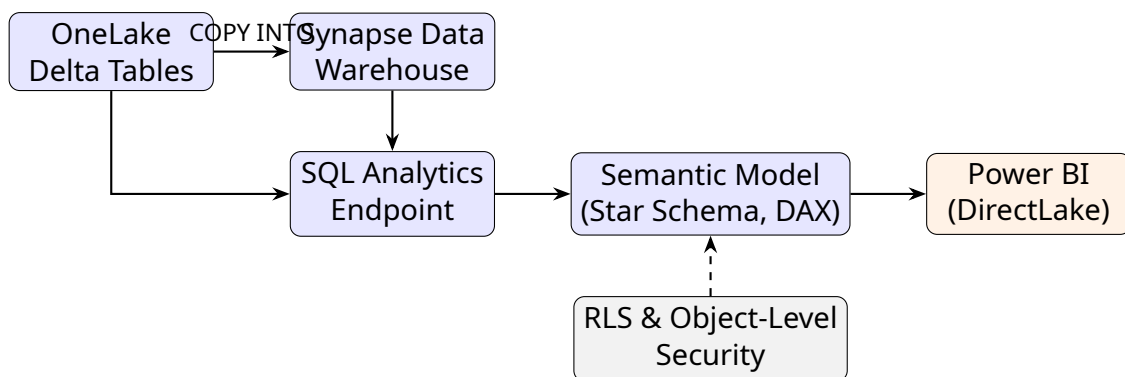


Figure 3: Milestone 2 reference architecture: warehouse-to-DirectLake serving layer

3 Milestone 3: Batch Processing & ETL Orchestration (Month 3)

Objective: Master Data Factory in Fabric, Dataflows Gen2, and Pipelines.

Learning outcomes: build metadata-driven pipelines, design Dataflows Gen2 with output destinations, configure hybrid gateways, orchestrate and monitor dependent workloads

Week 9: Data Factory Pipelines

Outcomes: build a metadata-driven pipeline with Lookup and ForEach, parameterize copies with dynamic expressions, enforce data-contract schema validation

- **Monday:** Fabric Pipelines: Copy Activity, Lookups, and Get Metadata.
- **Tuesday:** Control Flow: ForEach, If Condition, Until, and Switch.
- **Wednesday:** Parameters, Variables, and dynamic expressions in Pipelines. Data contracts: validate the expected schema in a notebook (or pipeline expression) before writing to Silver; Delta schema enforcement (default) vs. schema evolution (mergeSchema, overwriteSchema); breaking vs. non-breaking contract changes and contract versioning; who owns the contract — the producer workspace publishes and versions it, consumer workspaces validate against it.

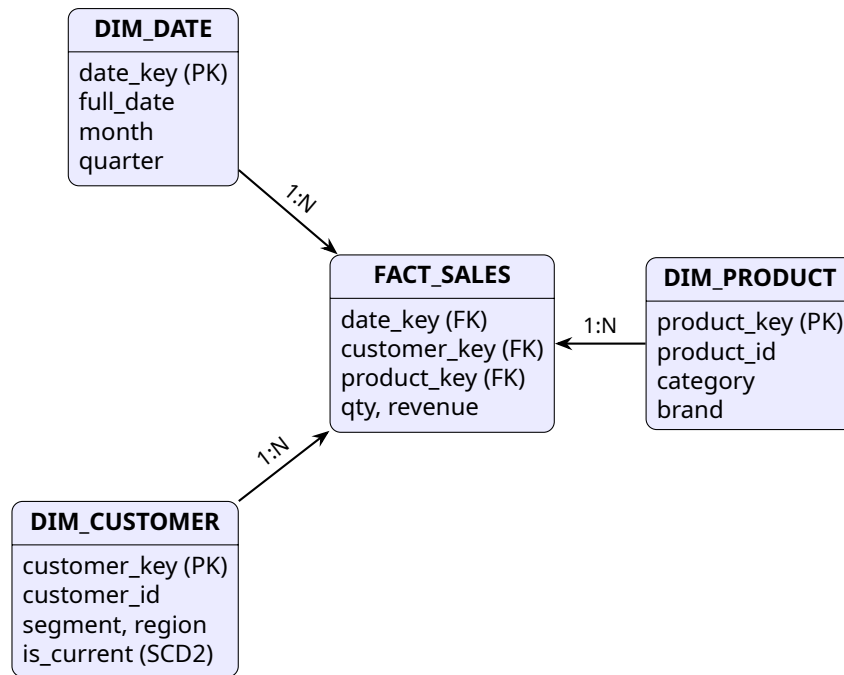


Figure 4: Capstone data model: warehouse star schema with conformed dimensions

- **Thursday: Hands-on Project:** Build a dynamic pipeline that looks up a list of table names from a configuration file and uses a ForEach loop to copy them from an Azure SQL DB to OneLake. Add an expected-schema validation step that rejects the batch when a source column is renamed or dropped (breaking change) but allows new additive columns (non-breaking).

```

// Pipeline JSON (abridged): @activity() feeds a ForEach,
// variables and dynamic expressions parameterize the Copy
{
  "name": "PL_Metadata_Ingest",
  "variables": { "vBatchTag": { "type": "String", "defaultValue": "daily" } },
  "activities": [
    { "name": "LookupTables", "type": "Lookup",
      "typeProperties": { "sqlReaderQuery":
        "SELECT schema_name, table_name FROM cfg.ingest_config WHERE is_active = 1" } },
    { "name": "CopyEachTable", "type": "ForEach",
      "dependsOn": [ { "activity": "LookupTables", "dependencyConditions": ["Succeeded"] } ],
      "typeProperties": {
        "items": "@activity('LookupTables').output.value",
        "activities": [ { "name": "CopyToOneLake", "type": "Copy",
          "typeProperties": {
            "source": { "sqlReaderQuery":
              "@concat('SELECT * FROM ', item().schema_name, '.', item().table_name)" },
            "sink": { "type": "LakehouseTable",
              "tableOption": "autoCreate",
              "partitionOption": "@concat(variables('vBatchTag'), '/',
                item().table_name)" } } } ] } } ] ] } } ] }
  }

```

- **Friday: Use Case & Architecture:** Design a metadata-driven ingestion framework capable of onboarding hundreds of new tables automatically without manual pipeline creation.

Resources: Data Factory in Fabric documentation (<https://learn.microsoft.com/fabric/data-factory/>); Apache Airflow repository (<https://github.com/apache/airflow>). **Self-check:**

- What makes an ingestion framework “metadata-driven”?
- Which activity iterates over a Lookup result set?
- What distinguishes a breaking from a non-breaking schema change?
- Who owns the data contract: the producer or the consumer workspace?
- What does `mergeSchema` allow that default Delta writes reject?

Week 10: Dataflows Gen2

Outcomes: transform API data with Dataflows Gen2 output destinations, configure incremental refresh, gate Gold writes with data-quality assertions and quarantine

- **Monday:** Power Query Online integration: Visual data transformations.
- **Tuesday:** Dataflows Gen2 output destinations (Lakehouse, Warehouse, Azure SQL).
- **Wednesday:** Staging vs. Data destinations, incremental refresh in Dataflows. Data-quality validation patterns: Great Expectations suites and notebook-based PySpark assertions gating writes to Gold.
- **Thursday: Hands-on Project:** Build a Dataflow Gen2 to connect to a web API, pivot and clean the JSON response using Power Query, and set the destination to a Gold Lakehouse table. Add a notebook data-quality assertion cell that gates the Gold write.

```
# Data-quality assertions before writing to Gold
from pyspark.sql import functions as F

df = spark.read.table("silver.orders")

null_pk = df.filter(F.col("order_id").isNull()).count()
dup_keys = (df.groupBy("order_id").count()
            .filter("count > 1").count())

assert null_pk == 0, f"DQ FAIL: {null_pk} null order_id values"
assert dup_keys == 0, f"DQ FAIL: {dup_keys} duplicate order_id keys"

# Quarantine bad rows instead of failing silently
rejected = df.filter(F.col("amount") < 0)
if rejected.count() > 0:
    rejected.withColumn("reject_reason", F.lit("negative_amount"))
              .write.format("delta").mode("append")
              .saveAsTable("gold.rejected_orders")

(df.filter(F.col("amount") >= 0)
 .write.format("delta").mode("overwrite")
 .saveAsTable("gold.orders"))
```

- **Friday: Use Case & Architecture:** Evaluate Mapping Dataflows (ADF), Dataflows Gen2, and Spark Notebooks for a team of business analysts migrating from Excel-based ETL.

Production Note: *Never fail silently on bad rows: quarantine records that violate expectations to a Rejected Delta table with an error-context column, alert via pipeline notification, and keep the Gold write clean. Failing the entire batch on a handful of bad rows breaks the SLA; ignoring them corrupts every downstream aggregate.*

Re-

sources: Dataflows Gen2 overview (<https://learn.microsoft.com/fabric/data-factory/dataflows-gen2>)
Great Expectations repository (https://github.com/great-expectations/great_expectations).

Self-check:

- What is an output destination in Dataflows Gen2?
- Why stage data before writing to a destination?
- What does incremental refresh avoid recomputing?
- Why quarantine bad rows instead of failing the whole batch?
- When are Spark notebooks preferable to Dataflows Gen2?

Week 11: Data Integration & Connectivity

Outcomes: connect on-premises and VNet sources via gateways, ingest REST APIs and webhooks, design a highly available gateway cluster

- **Monday:** On-premises Data Gateway vs. VNet Data Gateway.
- **Tuesday:** Connecting to hybrid sources (SQL Server, Oracle, SAP).
- **Wednesday:** Integrating REST APIs and Webhooks.
- **Thursday: Hands-on Project:** Configure a mock on-premises SQL Server. Install the On-premises Data Gateway and build a Fabric pipeline to extract data daily to OneLake.
- **Friday: Use Case & Architecture:** Architect a secure hybrid connectivity pattern for a bank that strictly forbids public IP endpoints for their on-premise transactional databases.

Production Note: *A single-node on-premises data gateway is a single point of failure for every hybrid pipeline. Install gateway clusters with two or more nodes for high availability and load balancing, keep gateway software patched within one release of current, and monitor node health centrally.*

Re-

sources: On-premises data gateway documentation (<https://learn.microsoft.com/data-integration/gateway/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>).

Self-check:

- When do you need a VNet data gateway instead of an on-premises gateway?
- Why cluster gateway nodes for production workloads?
- Which credential pattern should webhook/API ingestion use?
- How does a gateway keep data inside the bank's network perimeter?
- What breaks if the gateway machine goes offline mid-pipeline?

Week 12: Orchestration & Monitoring

Outcomes: orchestrate notebooks and Dataflows from pipelines, wire On Failure alerts with runbook links, monitor runs in the Monitoring Hub

- **Monday:** Invoking Notebooks and Dataflows from Pipelines.
- **Tuesday:** Dependency management (On Success, On Failure, On Completion).
- **Wednesday:** Monitoring Hub, alert routing, and Pipeline run history.
- **Thursday: Hands-on Project:** Create a master orchestration pipeline that triggers a Dataflow Gen2, waits for completion, triggers a Spark Notebook, and sends a Teams/Outlook alert on failure.

- **Friday: Use Case & Architecture:** Design a resilient, self-healing orchestration layer for a daily banking reconciliation process spanning 15 dependent systems using Fabric Pipelines.

Production Note: Route pipeline failures to Teams/Outlook and attach a runbook link to every alert: expected rerun steps, backfill procedure, and escalation owner. An alert without a runbook creates a 3 a.m. archaeology session. Use On Failure paths for compensating actions, not just notifications. Re-

sources: Monitoring hub documentation (<https://learn.microsoft.com/fabric/admin/monitoring-hub>); OpenLineage repository (<https://github.com/OpenLineage/OpenLineage>). **Self-check:**

- What is the difference between On Failure and On Completion paths?
- What does the Monitoring Hub show per pipeline run?
- Why attach a runbook link to every alert?
- How do you make a rerun of a failed batch idempotent?
- What is a compensating action in an On Failure path?

CAPSTONE PROJECT - Milestone 3: Automated Batch ELT Pipeline. Use Fabric Data Factory to build a complete pipeline. Ingest data via REST APIs using the Copy Activity, use Dataflows Gen2 for visual data cleansing, and orchestrate Spark notebooks for complex ML feature engineering. Schedule the pipeline and configure failure alerts.

Acceptance

- End-to-end pipeline
- Architecture diagram, runbook, and alerting
- At least one case study
- Monitoring and alerting
- Monthly vs. daily
- Security and hardening

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture:** config-table-driven pipeline (Lookup → ForEach Copy) + Dataflow Gen2 cleansing + Spark notebook feature engineering, orchestrated by a master pipeline with failure alerts; chosen so new tables onboard by inserting a config row, not by building pipelines.
- Create `cfg.ingest_config` with watermark and `is_active` columns.
- Build the dynamic pipeline with a schema-validation gate that rejects breaking contract changes.
- Land raw data in Bronze, cleanse via Dataflow Gen2 to Silver, run notebook DQ assertions with quarantine before Gold.
- Wire the master pipeline: On Failure → Teams/Outlook alert with runbook link; advance watermarks only on success.

```

-- Advance the watermark only after a successful batch
UPDATE cfg.ingest_config
SET watermark = @max_ts
WHERE schema_name = @sch AND table_name = @tbl
  AND @batch_status = 'Succeeded';
-- On rerun, the source query reads WHERE ts > @watermark,
-- so a retried batch never re-copies committed rows.

```

- **Pitfall:** non-idempotent reruns duplicate rows — watermark/manifest discipline is the fix.
- **Pitfall:** ForEach runs sequentially by default; enable parallelism with a bounded batch size.
- **Pitfall:** an alert without a runbook produces 3 a.m. archaeology — attach rerun and back-fill steps.

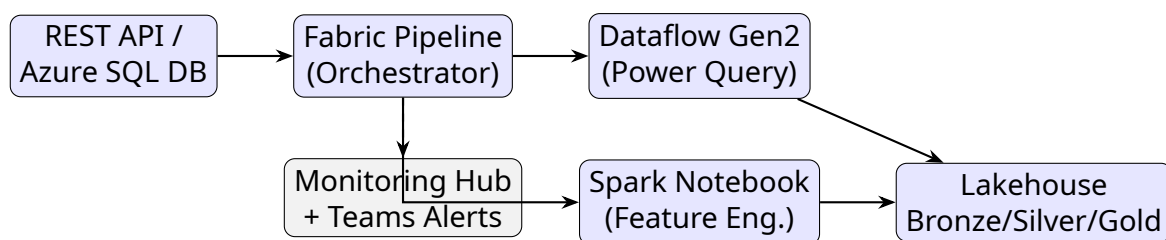


Figure 5: Milestone 3 reference architecture: orchestrated batch ELT

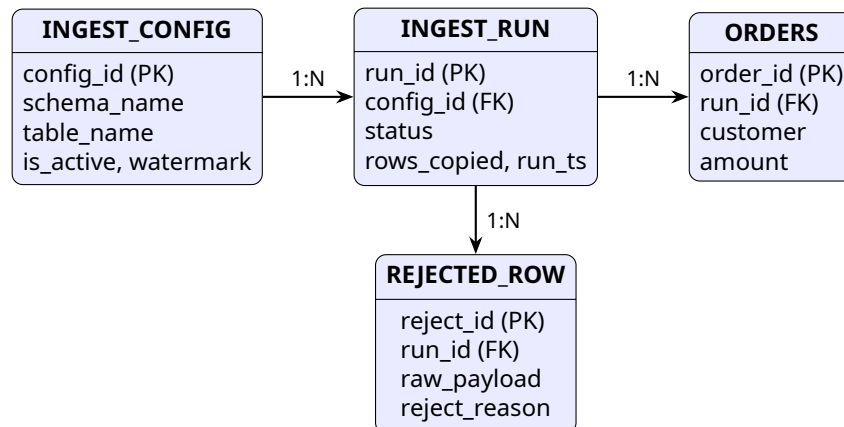


Figure 6: Capstone data model: metadata-driven batch ELT with quarantine

4 Milestone 4: Real-Time Intelligence (Month 4)

Objective: Conquer unbounded data, Eventstreams, and KQL Databases.

Learning outcomes: ingest streaming events with Eventstreams, query time series with KQL, detect anomalies, automate alerts with Data Activator, build real-time dashboards

Week 13: Eventstreams & Real-Time Ingestion

Outcomes: ingest mock IoT events with Eventstreams, route to KQL and Lakehouse destinations, design idempotent replay-safe consumers

- **Monday:** Real-Time Intelligence architecture in Fabric.
- **Tuesday:** Eventstreams: Sources (Azure Event Hubs, Custom Apps, IoT).
- **Wednesday:** Eventstream Destinations (Lakehouse, KQL Database, Reflex).
- **Thursday: Hands-on Project:** Create a Fabric Eventstream. Write a Python script to send mock IoT temperature data to the custom app endpoint, and route the data to a KQL Database.
- **Friday: Use Case & Architecture:** Architect a global telemetry ingestion point for 5 million mobile games, ensuring high throughput and low-latency processing into Fabric.

Production Note: *Design Eventstream consumers for idempotency and replay: events can be delivered more than once, so stamp every event with a unique ID and dedupe downstream (e.g., Delta MERGE on event_id). Know the source's replay window (Event Hubs retention) so you can reprocess after a downstream failure without data loss or duplicates.*

Re-

sources: Fabric Eventstreams documentation (<https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview>); Apache Kafka repository (<https://github.com/apache/kafka>). **Self-check:**

- Name two Eventstream sources and one destination.
- Why stamp every event with a unique ID?
- What does the Event Hubs retention (replay) window control?
- When would you route events to a KQL database vs. a Lakehouse?
- What does at-least-once delivery imply for downstream consumers?

Week 14: KQL Databases (Eventhouses)

Outcomes: write KQL time-series queries with make-series, compute rolling averages, detect anomalies with series functions, maintain incremental state with materialized views, deduplicate replayed events with arg_max, and join streams to streams and dimensions with time-windowed keys

- **Monday:** Eventhouses and Kusto Query Language (KQL) databases.
- **Tuesday:** KQL syntax basics: search, where, project, summarize. Materialized views as incremental state over append-only streams; event deduplication with arg_max on event id and ingestion_time().

- **Wednesday:** Advanced KQL: time series analysis, stream-to-stream and stream-to-dimension joins, time-windowed join keys, late-arrival implications for windowed aggregates, and why unbounded lookback windows degrade query and materialization performance.
- **Thursday: Hands-on Project:** Query the IoT data in your KQL database. Write a KQL query to calculate the 5-minute rolling average temperature and detect anomalies using built-in functions.

```
// 5-minute rolling average + anomaly detection per device
IoTTelemetry
| where Timestamp > ago(6h)
| make-series AvgTemp = avg(Temperature) default=0
    on Timestamp step 5m by DeviceId
| extend RollingAvg = series_fir(AvgTemp, repeat(1, 5))
| extend Anomalies = series_decompose_anomalies(RollingAvg, 1.5)
| mv-expand Timestamp, RollingAvg, Anomalies
| project DeviceId,
    Timestamp = todatetime(Timestamp),
    RollingAvg = todouble(RollingAvg),
    IsAnomaly = toint(Anomalies) == 1
| order by DeviceId, Timestamp
```

- **Friday:** End-to-end exactly-once: Eventstream at-least-once delivery, idempotent ingestion into KQL/Lakehouse, dedup-key placement, and downstream consumer checkpointing. **Use Case & Architecture:** Troubleshoot a scenario where a streaming dashboard is showing inaccurate daily aggregations due to mobile devices going offline and uploading data late.

```
// Dedup (keep latest arrival per id), windowed stream join, dimension lookup
let Telemetry =
    IoTTelemetry
    | where Timestamp > ago(1h) // bounded lookback window
    | summarize arg_max(ingestion_time(), *) by EventId;
let Alerts =
    DeviceAlerts
    | where AlertTime > ago(1h)
    | summarize arg_max(ingestion_time(), *) by AlertId;
Telemetry
| join kind=inner Alerts on DeviceId
| where abs(datetime_diff('second', Timestamp, AlertTime)) <= 300
| join kind=inner ( // stream-to-dimension
    Devices | project DeviceId, Site, Model
) on DeviceId
| summarize Readings = count() by Site, Model, bin(Timestamp, 1m)
```

Production Note: Eventstream replay is the top source of silent duplicates downstream: after a consumer failure or reconfiguration, Eventstream redelivers events from the source's retention window, and every at-least-once hop (Eventstream → KQL, Eventstream → Lakehouse) can double-land rows. Stamp the dedup key at the producer (a unique *event_id* generated before the first send, not at any hop), propagate it untouched through Eventstream, and enforce dedup at the first queryable boundary — a KQL materialized view with *arg_max*, or a Delta MERGE on *event_id* for the Lakehouse — so every downstream consumer (dashboards, Activator rules, reverse ETL) reads already-clean data. Pair this with consumer checkpointing: record the last processed event time/offset so restarts resume instead of replaying.

Re-

sources: Kusto Query Language (KQL) documentation (<https://learn.microsoft.com/azure/data-explorer/kusto/query/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- How does a KQL materialized view maintain incremental state over an append-only stream, and what does `arg_max by EventId with ingestion_time()` deduplicate?
- Why do unbounded lookback windows degrade query and materialization performance?
- How do you express a time-windowed stream-to-stream join in KQL, and how do late arrivals affect it?
- Where must the dedup key be generated and enforced for end-to-end exactly-once semantics under Eventstream replay?
- What does downstream consumer checkpointing protect against, and what breaks without it?

Week 15: Data Activator (Reflex)

Outcomes: build Data Activator triggers with sustained-threshold conditions, automate Teams and email actions, suppress alert noise

- **Monday:** Data Activator concepts: Triggers, Conditions, and Actions.
- **Tuesday:** Creating Reflex items from Power BI visuals or Eventstreams.
- **Wednesday:** Automated actions: Teams messages, Emails, and custom workflows.
- **Thursday: Hands-on Project:** Set up a Data Activator trigger on your streaming IoT data. Configure an alert to send an email if the temperature exceeds a threshold for 3 consecutive minutes.
- **Friday: Use Case & Architecture:** Design a proactive alerting system for a logistics company to instantly notify drivers via custom app webhooks when a delivery truck goes off route.

Resources: Data Activator (Reflex) documentation (<https://learn.microsoft.com/fabric/real-time-intelligence/data-activator/activator-introduction>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- Which three parts define a Data Activator trigger?
- Why require “3 consecutive minutes” instead of a single spike?
- Which actions can Reflex fire natively?
- How do you create an alert from a Power BI visual?
- What alert latency should you expect from a Reflex item?

Week 16: Real-Time Dashboards

Outcomes: build auto-refreshing real-time dashboards on KQL, implement reverse-ETL upserts to Dataverse on stable business keys, weigh sync frequency against CU cost

- **Monday:** KQL Querysets vs. Real-Time Dashboards.
- **Tuesday:** Building visual dashboards directly on KQL databases.
- **Wednesday:** Power BI automatic page refresh (DirectQuery vs KQL). Reverse ETL: pushing curated Gold Lakehouse/warehouse data back to Dataverse or an operational CRM via pipelines — identity resolution (matching on stable business keys such as Dataverse IDs), and sync-frequency trade-offs (near-real-time micro-batches vs. nightly full syncs, and their CU/API-throttling cost).

- **Thursday: Hands-on Project:** Create a Real-Time Dashboard natively in Fabric using KQL queries. Add auto-refreshing charts for active IoT devices and current temperature trends. Then build a reverse-ETL pipeline that upserts a Gold customer-segment table back into Dataverse, keyed on a stable business key.
- **Friday: Use Case & Architecture:** Design the BI access layer for a manufacturing floor where operators need sub-second latency dashboards monitoring machine health (vibration and heat).

Resources: Real-Time Dashboards documentation (<https://learn.microsoft.com/fabric/real-time-intel-dashboard-real-time-create>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- When would you choose a Real-Time Dashboard over a KQL queryset?
- What does automatic page refresh cost in DirectQuery mode?
- What is reverse ETL, and why push Gold data back to a CRM?
- Which identity-resolution key would you sync Dataverse records on?
- Trade-off between 15-minute and nightly reverse-ETL syncs?

CAPSTONE PROJECT - Milestone 4: Real-Time Fraud Detection. Build a Python script simulating credit card swipes sent to a Fabric Eventstream. Route data to a KQL Database. Write KQL queries to detect if a card is used in two different cities within 10 minutes. Use Data Activator to instantly alert the security team on Microsoft Teams.

Acceptance Criteria

- End-to-end pipeline: Python producer → Eventstream → Eventhouse/KQL → impossible-travel fraud query → Data Activator → Teams alert
- Architecture: Event-driven, Scalable, Reflexive
- At least 1000 test cases with a coverage of 100%
- Monitoring: Real-time replay, Alerting
- Monthly security audit and decision
- Security: KQL access control, Data Activator permissions

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture:** Python producer (faker) → Eventstream → Eventhouse/KQL → impossible-travel fraud query → Data Activator → Teams alert; chosen for sub-minute detection on unbounded event streams.
- Generate card swipes with `faker`, including deliberate two-cities-in-10-minutes cases, and stamp every event with a unique `txn_id`.
- Route the Eventstream to a KQL database; deduplicate on `txn_id` downstream (at-least-once delivery).

- Implement the fraud rule in KQL and create a Reflex trigger with a sustained condition to avoid alert storms.
- Prove replay: reprocess from the Event Hubs retention window after a simulated downstream outage.

```
// Impossible travel: same card, different cities, <= 10 min apart
Txn
| order by card_id asc, ts asc
| serialize
| extend p_card = prev(card_id), p_city = prev(city), p_ts = prev(ts)
| where card_id == p_card and city != p_city and ts - p_ts <= 10m
| project card_id, ts, city, p_city
```

- **Pitfall:** late-arriving events shift window aggregates — define and document a lateness policy.
- **Pitfall:** duplicates from at-least-once delivery double-fire alerts; dedupe before the rule query.
- **Pitfall:** single-spike triggers alert-storm the team; require sustained conditions (e.g., 3 consecutive minutes).

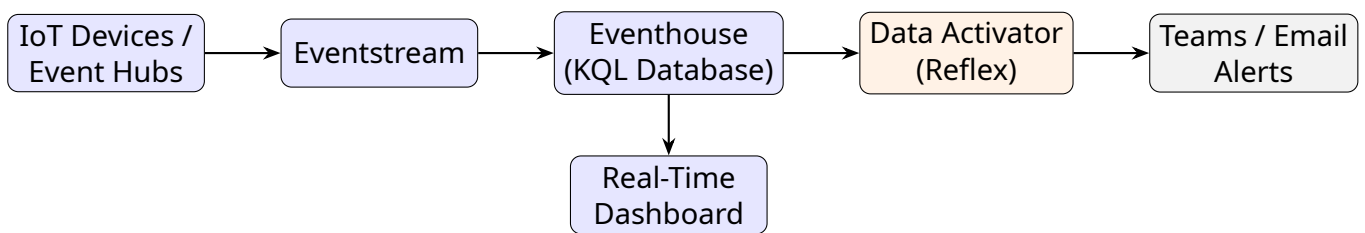


Figure 7: Milestone 4 reference architecture: real-time intelligence pipeline

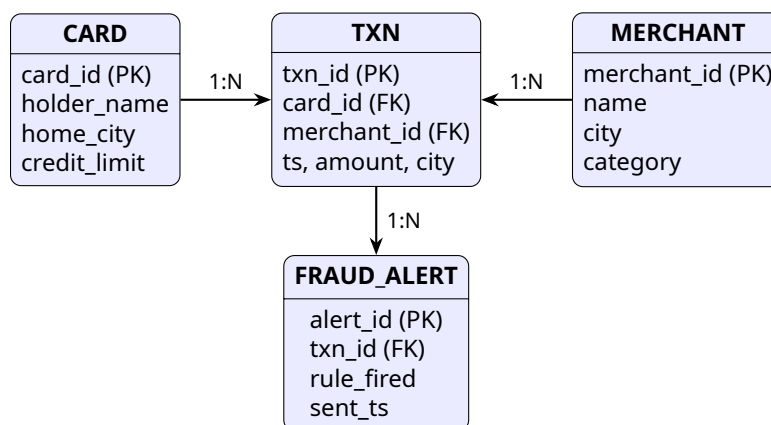


Figure 8: Capstone data model: real-time fraud detection

5 Milestone 5: Data Science & AI Operationalization (Month 5)

Objective: Embed AI into data pipelines using MLflow and SynapseML.

Learning outcomes: prepare features with Data Wrangler, track experiments with MLflow, enrich data with SynapseML and Azure OpenAI, operationalize batch scoring

Week 17: Data Science Foundations in Fabric

Outcomes: clean data visually with Data Wrangler and export PySpark, perform EDA on samples before full passes, engineer leakage-safe features

- **Monday:** The Data Science experience: Notebooks, Data Wrangler.
- **Tuesday:** Exploratory Data Analysis (EDA) with Pandas and PySpark.
- **Wednesday:** Feature engineering and preparation for machine learning.
- **Thursday: Hands-on Project:** Use Fabric Data Wrangler in a Notebook to visually clean a dataset, handle missing values, and generate the corresponding PySpark code automatically.
- **Friday: Use Case & Architecture:** Architect a system to automatically forecast daily inventory needs for 1,000 retail stores using purely Spark-based pipelines.

Resources: Fabric Data Science documentation (<https://learn.microsoft.com/fabric/data-science/>); Apache Spark repository (<https://github.com/apache/spark>). **Self-check:**

- What does Data Wrangler generate from your visual cleaning steps?
- Why run EDA on a sample before a full PySpark pass?
- Name two leakage risks in feature engineering.
- Why keep feature sets in the Lakehouse instead of CSV exports?
- What makes a forecasting pipeline “Spark-native”?

Week 18: Model Training & MLflow

Outcomes: track experiments with mlflow.autolog, register and version models, compare hyperparameter runs by metric

- **Monday:** Machine Learning Experiments and Models in Fabric.
- **Tuesday:** Using MLflow for tracking parameters, metrics, and artifacts.
- **Wednesday:** Registering models in the Fabric Model Registry.
- **Thursday: Hands-on Project:** Train a Scikit-Learn Random Forest model. Use ‘mlflow.autolog()’ to automatically track the experiment in Fabric, and register the best model.
- **Friday: Use Case & Architecture:** Design a workflow for data scientists to track hundreds of hyperparameter tuning runs collaboratively within a shared Fabric Workspace.

Resources: MLflow in Fabric documentation (<https://learn.microsoft.com/fabric/data-science/mlflow-overview>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>)

Self-check:

- What does `mlflow.autolog()` capture per run?
- Why register a model instead of referencing run artifacts directly?
- Which metadata should accompany every experiment run?
- How do you compare hundreds of hyperparameter runs efficiently?
- What does model versioning enable for rollback?

Week 19: SynapseML & AI Integration

Outcomes: enrich columns with SynapseML and Azure OpenAI in batch, keep API keys out of notebooks, publish sentiment to Gold for BI

- **Monday:** Introduction to SynapseML (Distributed ML library).
- **Tuesday:** Pre-built AI Models: Integrating Azure OpenAI and Cognitive Services.
- **Wednesday:** Copilot in Fabric: Generating code and insights using natural language.
- **Thursday: Hands-on Project:** Use SynapseML in a Spark notebook to call the Azure OpenAI API. Pass a column of customer reviews and append a new column with the sentiment analysis result.
- **Friday: Use Case & Architecture:** Design a scalable pipeline to process terabytes of uploaded user reviews, extract sentiment, and make the metadata available in a Gold Lakehouse.

Resources: SynapseML repository (<https://github.com/microsoft/SynapseML>); Azure OpenAI in Fabric docs (<https://learn.microsoft.com/fabric/data-science/ai-services/overview>). **Self-check:**

- What does SynapseML add on top of Spark MLlib?
- How do you avoid hardcoding Azure OpenAI keys in a notebook?
- Why batch sentiment calls instead of calling the API row-by-row?
- What can Copilot in Fabric generate from natural language?
- Where should sentiment results land to be usable by BI?

Week 20: MLOps & Batch Scoring

Outcomes: batch-score Delta tables with the PREDICT function, automate inference in pipelines, detect data drift and trigger retraining

- **Monday:** Batch scoring with Spark 'PREDICT' function.
- **Tuesday:** Applying registered MLflow models to Delta tables.
- **Wednesday:** Automating model inference via Fabric Pipelines.
- **Thursday: Hands-on Project:** Load a registered MLflow model and use the 'PREDICT' function in Spark to score a new batch of 100,000 rows. Write the predictions back to OneLake.
- **Friday: Use Case & Architecture:** Architect an MLOps lifecycle that detects data drift in a production recommendation engine and automatically triggers a retraining Pipeline.

Resources: Machine learning model training in Fabric (<https://learn.microsoft.com/fabric/data-science/model-training-overview>); MLflow repository (<https://github.com/mlflow/mlflow>). **Self-check:**

- What does the Spark PREDICT function do?
- Why score in scheduled batch instead of per-request here?
- Which signal indicates data drift in production?

- Why automate inference via pipelines instead of manual notebook runs?
- Which schema contract must scoring output honor downstream?

CAPSTONE PROJECT - Milestone 5: End-to-End MLOps Pipeline. Use Data Factory Pipelines to orchestrate a workflow that extracts features from a Lakehouse, triggers a Notebook to train a churn prediction model, tracks it with MLflow, registers the model, and runs a daily batch scoring Notebook that populates a Power BI DirectLake report.

Acceptance Criteria

- End-to-end pipeline → reproducible
- Architecture, data flow, and schema
- At least one schema contract
- Monitoring and alerting
- Monthly window
- Security and compliance noted.

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture:** Gold feature table → MLflow-tracked training notebook → model registry → scheduled batch-scoring notebook → Power BI DirectLake report; chosen to keep the full MLOps loop inside one Fabric workspace.
- Build leakage-safe features in Gold (no post-outcome columns; fixed training windows).
- Train with `mlflow.autolog()`; register the best run with dataset version and code commit as metadata.
- Orchestrate scoring via a pipeline; write predictions with a `model_version` column for auditability.
- Add a drift check comparing feature distributions against training baselines; trigger re-training on breach.

```
# Batch scoring with the registered model (Spark UDF at scale)
import mlflow
from pyspark.sql import functions as F

predict = mlflow.pyfunc.spark_udf(spark, "models:/churn_model/Production",
                                   result_type="double")
df = spark.read.table("gold.churn_features")
(df.withColumn("churn_score", predict(*df.columns))
 .withColumn("model_version", F.lit("Production")))
.write.format("delta").mode("append")
.saveAsTable("gold.churn_predictions"))
```

- **Pitfall:** training/serving skew — compute scoring features with the same logic used in training.

- **Pitfall:** predictions without `model_version` cannot be audited or rolled back.
- **Pitfall:** drift unmonitored means the model quietly rots; alert on distribution shift, not just errors.

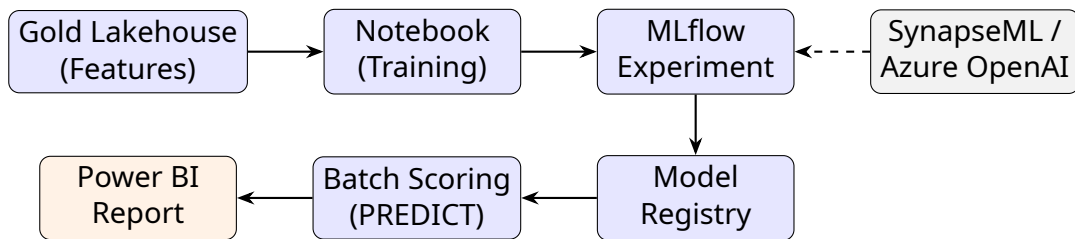


Figure 9: Milestone 5 reference architecture: MLflow-driven MLOps lifecycle

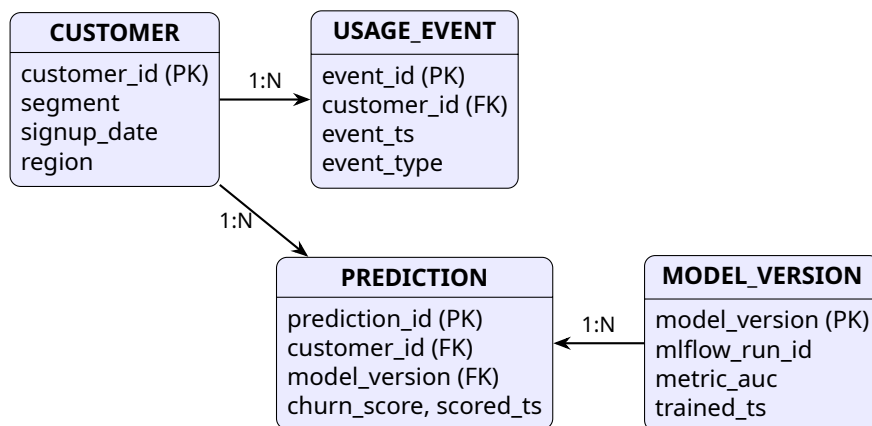


Figure 10: Capstone data model: churn MLOps (features, models, predictions)

6 Milestone 6: Security, Administration & Exam Prep (Month 6)

Objective: Secure OneLake, implement Purview, and prepare for DP-600.

Learning outcomes: apply least-privilege access controls, govern data with Purview labels and lineage, implement Git-based CI/CD with deployment pipelines, pass the DP-600 exam

Week 21: Security & Access Control

Outcomes: assign least-privilege workspace and item roles, configure OneLake data access roles and SQL RLS, audit for privilege creep

- **Monday:** Workspace Roles: Admin, Member, Contributor, Viewer.
- **Tuesday:** Item-level permissions vs. Workspace-level permissions.
- **Wednesday:** OneLake Data Access Roles (Preview) and Row-Level Security (RLS) in SQL.
- **Thursday: Hands-on Project:** Create a Workspace and assign a Contributor role. Configure Row-Level Security on a SQL Analytics Endpoint to restrict a user to seeing only 'US' region data.
- **Friday: Use Case & Architecture:** Design a highly secure data perimeter for a healthcare provider ensuring strict separation of duties between Data Engineers and Data Analysts within Fabric.

Production Note: *Grant access at the narrowest scope: prefer item-level permissions and OneLake data access roles over workspace-wide Member/Admin assignments. Audit quarterly — Viewer on a workspace combined with broad item sharing is the most common form of privilege creep in Fabric tenants.*

Re-

sources: Fabric security documentation (<https://learn.microsoft.com/fabric/security/>); DataHub repository (<https://github.com/datahub-project/datahub>). **Self-check:**

- What distinguishes Contributor from Viewer workspace roles?
- Why prefer item-level permissions over workspace-level grants?
- What do OneLake data access roles control?
- How does SQL RLS restrict a user to 'US' region rows only?
- What is privilege creep, and how do you audit for it?

Week 22: Governance, Purview & GDPR Erasure

Outcomes: apply and inherit sensitivity labels, trace end-to-end lineage for impact analysis, execute a GDPR erasure request via crypto-shredding across Delta, KQL, and caches

- **Monday:** Purview Hub in Fabric and tenant-level administration.
- **Tuesday:** Information Protection: Sensitivity labels applied to Fabric items; label inheritance downstream and the interaction with legal-hold/retention policies — a legal hold overrides an erasure request and must be recorded as a documented exception in the audit register.

- **Wednesday:** Data Lineage, Endorsement (Promoted/Certified), and Impact Analysis. GDPR right-to-erasure propagation path: a deletion request must reach OneLake raw zones, Bronze/Silver/Gold Delta tables (including Delta time-travel history files and the VACUUM implications for purging them), Eventstream/KQL retention windows, downstream derived tables, semantic model caches, and clones/backups. Compare crypto-shredding (per-subject encryption keys in Azure Key Vault — deleting the key erases every copy including backups) against row-level DELETE with downstream replay.
- **Thursday: Hands-on Project:** Apply a 'Highly Confidential' sensitivity label to a Lakehouse. View the automated data lineage graph from the Lakehouse through a Pipeline to a Power BI report. Then implement crypto-shredding: generate per-subject encryption keys in Azure Key Vault, encrypt PII columns in a notebook, and process a sample erasure request by deleting one subject's key — verifying the subject's data becomes unrecoverable in current tables and Delta history alike, while a row-level DELETE leaves rows readable via time travel until VACUUM runs.
- **Friday: Use Case & Architecture:** Architect a Data Mesh for a global conglomerate with 5 independent domains, ensuring centralized governance via Purview while using separate Fabric Workspaces. Add the erasure-propagation runbook each domain must implement for its own Bronze/Silver/Gold Lakehouses and KQL databases.

Production Note: *Naive DELETE is insufficient for GDPR erasure in Fabric: Delta time travel keeps deleted rows readable in history files until VACUUM expires them, Eventstream/KQL data persists per its retention policy, semantic model caches hold copies in memory, and clones/backups re-create erased rows on restore. Crypto-shredding sidesteps all of these — deleting the per-subject key in Azure Key Vault renders every encrypted copy unreadable, including history and backups. Coordinate VACUUM retention, KQL retention policies, and the legal-hold register so erasure windows and litigation exceptions do not conflict.*

Re-

sources: Microsoft Purview and Fabric documentation (<https://learn.microsoft.com/fabric/governance/microsoft-purview-fabric>); DataHub repository (<https://github.com/datahub-project/datahub>). **Self-check:**

- What does a sensitivity label pass on to downstream items?
- What is the difference between Promoted and Certified endorsement?
- How does lineage help with impact analysis before a change?
- What does the Purview Hub surface to tenant administrators?
- Why separate workspaces per domain in a data-mesh design?

Week 23: Administration & CI/CD

Outcomes: analyze capacity metrics and throttling, run Git-integrated CI/CD with deployment pipelines, deploy zero-downtime expand-contract schema changes

- **Monday:** Fabric Capacity management, metrics app, and pausing/resuming. Hands-on: open the Fabric Capacity Metrics app against a loaded capacity, identify throttling events (interactive vs. background rejection, smoothing windows), and record the CU consumption trend to decide when to scale up.
- **Tuesday:** Git Integration: Syncing Workspaces with Azure DevOps/GitHub. Disaster-recovery role: Git Integration plus Deployment Pipelines serve as the recovery mechanism for workspace items — Fabric has no traditional backup/restore for workspace metadata.

- **Wednesday:** Deployment Pipelines: Dev → Test → Prod promotion. Designing multi-region BCDR: since OneLake data is not automatically replicated across regions, plan deliberate dual-workspace / dual-capacity designs with data re-ingestion or Delta clone strategies. Zero-downtime schema evolution (expand → migrate → contract): add the new nullable column, dual-write and backfill it, switch consumers to the new column behind backward-compatible view shims on the SQL analytics endpoint, then drop the old column in a later release — sequenced through Git Integration and Deployment Pipelines so schema-change notebooks deploy before the pipelines, notebooks, and semantic models that depend on them (the Fabric equivalent of coordinating dbt/schemachange migration order).
- **Thursday: Hands-on Project:** Connect your Workspace to an Azure DevOps repository. Commit your Notebooks and Pipelines. Create a Deployment Pipeline to promote items to a 'Prod' workspace.
- **Friday: Use Case & Architecture:** Design an enterprise CI/CD pipeline allowing multiple data engineering teams to collaborate on the same Lakehouse without overwriting each other's code.

Production Note: *Fabric DR reality check: OneLake has no traditional backup/restore. Workspace items (notebooks, pipelines, models) are recoverable because Git Integration + Deployment Pipelines rehydrate them; OneLake data recovery relies on Delta time travel and soft-delete patterns, and multi-region BCDR requires a deliberate dual-workspace design — there is no automatic cross-region failover today.*

Production
ing on CU
pause non
users comp

Production Note: *Failure story (deploy order): a deployment pipeline promoted a notebook that renamed `customer_segment` before the Gold view shim was deployed; the DirectLake semantic model still referenced the old column and every executive dashboard errored at 8 a.m. The fix was to treat view shims as a separate, always-first deployment stage in the pipeline, and to drop old columns only after one full release cycle of dual-running old and new columns.*

Re-

sources: Fabric Git integration documentation (<https://learn.microsoft.com/fabric/cicd/git-integration/intro-to-git-integration>); Fabric Capacity Metrics app (<https://learn.microsoft.com/fabric/enterprise/metrics-app>). **Self-check:**

- Which CU consumption behavior triggers background throttling?
- Why is Git Integration the recovery mechanism for workspace items?
- Why is there no automatic cross-region failover for OneLake?
- What does surge protection guard against?
- Why pause non-production capacities on a schedule?

Week 24: DP-600 Certification Preparation

Outcomes: assess readiness against DP-600 skill domains, remediate weak domains with targeted review, pass two full-length mock exams

- **Monday:** Review DP-600 (Implementing Analytics Solutions Using Microsoft Fabric) Objectives.
- **Tuesday:** Deep dive: DirectLake vs Import vs DirectQuery optimization.
- **Wednesday:** Full-length Mock Exam 1 (Identify weak domains).

- **Thursday:** Targeted review of weak domains (e.g., Spark optimization, Pipeline parameters).
- **Friday:** Full-length Mock Exam 2.

Resources: DP-600 certification and study guide (<https://learn.microsoft.com/credentials/certifications/fabric-analytics-engineer-associate/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- Which skill domains does the DP-600 exam weight most heavily?
- When does DirectLake beat Import in exam scenarios?
- How do you triage weak domains after Mock Exam 1?
- Which capacity metric signals DirectLake fallback?
- What score thresholds mark you certification-ready in this program?

CAPSTONE PROJECT - Milestone 6: Secure, Governed Data Platform. Build a multi-workspace architecture. Ingest streaming PII data. Use Dataflows Gen2 to mask the PII. Apply Sensitivity Labels and Endorsements. Connect to Azure DevOps and use Deployment Pipelines to seamlessly promote the architecture from a Development Workspace to Production.

Acceptance Criteria

- Full platform deployment
- Architecture in Purview
- At least all environments
- Monitoring with runbooks
- Monthly security protection
- Security no security workspace

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture:** Dev/Test/Prod workspaces with Git Integration + Deployment Pipelines, Purview sensitivity labels and lineage, PII masked before Gold; chosen so the whole platform redeploys from Git and governs itself by label inheritance.
- Ingest synthetic PII streams in Dev; mask PII in a Dataflow Gen2 (or notebook) before any Gold write.
- Apply sensitivity labels and endorsements; verify downstream inheritance in the Purview Hub.
- Connect the workspace to Azure DevOps; promote via a Deployment Pipeline with deployment rules per stage.
- Enforce least-privilege roles plus SQL RLS; document the DR plan (Git recovery + Delta time travel + dual-workspace BCDR).

```
# Mask PII before Gold: hash emails, redact phone numbers
from pyspark.sql import functions as F

masked = (df
    .withColumn("email", F.sha2(F.col("email"), 256))
    .withColumn("phone", F.regexp_replace("phone", r"\d", "*")))

(masked.write.format("delta").mode("overwrite")
    .saveAsTable("gold.patients_masked"))
```

- **Pitfall:** workspace items sync to Git verbatim — secrets in a notebook are secrets in the repo.
- **Pitfall:** labels are not automatically applied to every downstream item; audit inheritance in Purview.
- **Pitfall:** deploy-order breaks consumers; ship view shims first and drop old columns a release later (Week 23).

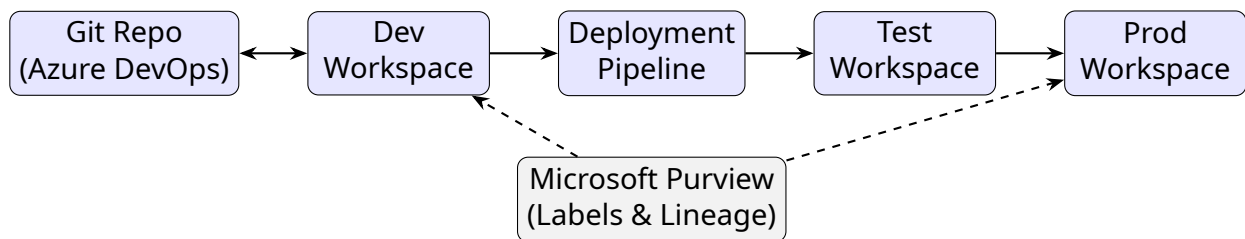


Figure 11: Milestone 6 reference architecture: governed CI/CD platform

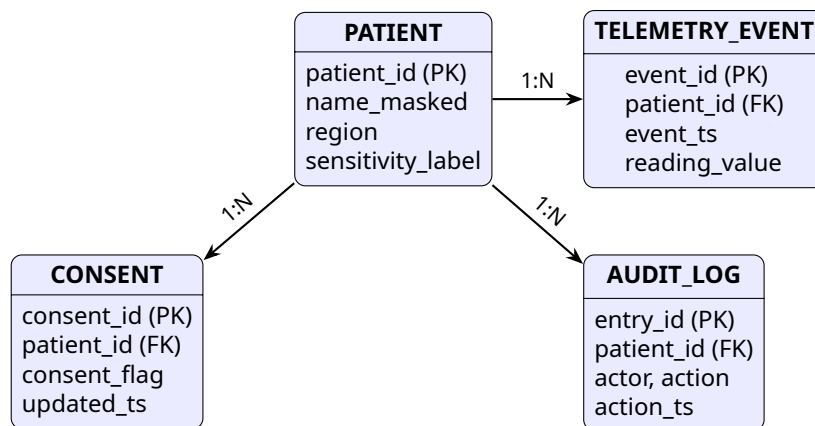


Figure 12: Capstone data model: governed PII streaming platform

7 Appendix A: Glossary

OneLake	The single, tenant-wide data lake underlying all of Fabric (“OneDrive for data”).
Lakehouse	A Fabric item combining files and managed Delta tables with a SQL analytics endpoint.
Data Warehouse	Fabric’s T-SQL-native analytical store with full transactional DDL/DML support.
Delta Lake	Open table format (Parquet + transaction log) used by all Fabric engines.
Medallion Architecture	Bronze (raw) → Silver (validated) → Gold (enriched) layering pattern.
Shortcut	A OneLake pointer that virtualizes external data (ADLS, S3, Dataverse) without copying it.
Semantic Model	The Power BI dataset layer defining tables, relationships, and DAX measures.
DirectLake	Power BI storage mode reading Delta/Parquet directly into memory without import or DirectQuery.
SQL Analytics Endpoint	The automatic read-only T-SQL endpoint generated for every Lakehouse.
Dataflow Gen2	Power Query Online-based visual ETL with output destinations.
Pipeline	Fabric Data Factory orchestration item (Copy, ForEach, Notebook activities, etc.).
Eventstream	Real-time ingestion and routing item for streaming events.
Eventhouse	Fabric’s real-time analytics store hosting one or more KQL databases.
KQL	Kusto Query Language for time-series and log analytics over Eventhouses.
Data Activator (Reflex)	No-code item that triggers actions when data meets conditions.
MLflow	Open-source experiment tracking and model registry framework integrated into Fabric.
SynapseML	Microsoft’s distributed ML library for Spark, including Azure AI service connectors.
Capacity Unit (CU)	Fabric’s compute billing unit; capacities are sized in SKUs (F2–F2048).
Throttling	Capacity protection that delays or rejects operations when CU consumption exceeds limits.
Workspace Role	Admin, Member, Contributor, or Viewer — the coarse access boundary in Fabric.
Git Integration	Native sync of workspace items to Azure DevOps/GitHub repositories.

Deployment Pipeline

Fabric item promoting content across Dev → Test → Prod workspaces.

Sensitivity Label

Microsoft Purview classification applied to Fabric items, inherited downstream.

Monitoring Hub

Central Fabric surface for run history, status, and alert routing.

Purview Hub

In-Fabric governance portal for lineage, endorsement, and tenant insights.

8 Appendix B: Assessment & Grading Model

Each monthly capstone is graded against its Acceptance Criteria box using the following weights:

Category	Weight
Correctness (solution runs end-to-end with accurate results)	30%
Reliability & tests (data-quality tests, idempotency, retries)	20%
Security (least-privilege, no secrets in code, PII handling)	15%
Cost awareness (CU/capacity estimate and optimizations)	15%
Documentation (README, architecture diagram, runbook)	10%
Demo (live walkthrough of the working solution)	10%

The two DP-600 mock exams in Week 24 feed the final readiness score: “certification ready” requires an average capstone grade of at least 80% *and* a mock-exam average of at least 75%, with no single weak domain left unremediated.

9 Appendix C: Interview Preparation

1. When would you use a OneLake shortcut instead of copying data, and what are the trade-offs? (Week 1)
2. How do you decide between a Lakehouse and a Data Warehouse for a given workload? (Weeks 2, 5)
3. A Delta table has degraded badly over months of streaming — how do you diagnose and fix it? (Week 4)
4. Explain DirectLake vs. Import vs. DirectQuery, and how you detect fallback in production. (Weeks 8, 24)
5. Design a metadata-driven ingestion framework that onboards new tables without new pipelines. (Week 9)
6. How do you enforce data quality before Gold writes without failing entire batches on a few bad rows? (Week 10)
7. A pipeline failed at 3 a.m. — walk through your alerting, runbook, and rerun strategy. (Week 12)
8. How do you handle late-arriving and duplicate events in a KQL/Eventstream pipeline? (Weeks 13–14)

9. How would you track, register, and batch-score an ML model entirely inside Fabric? (Weeks 18, 20)
10. Fabric has no traditional backup/restore — design the DR and multi-region BCDR strategy. (Week 23)

10 Appendix D: Self-Check Answer Key

Week 1:

1. A shortcut virtualizes external data in place; no copy or duplication.
2. Capacity provides compute CUs; a workspace organizes items and access.
3. ADLS Gen2 and Amazon S3 (also Dataverse, Google Cloud Storage).
4. Microsoft manages infrastructure and updates; you consume capacities, not servers.
5. Domains group workspaces by business area for delegated governance.

Week 2:

1. Bronze holds raw unvalidated data; Silver holds cleaned, conformed data.
2. When T-SQL DDL/DML, transactions, or stored procedures are required.
3. Isolation of access, lifecycle, and capacity billing per layer.
4. The Delta transaction log alongside Parquet data files.
5. Only curated-output owners; Gold feeds reports, so writes stay controlled.

Week 3:

1. Starter pools pre-warm shared nodes; custom pools are sized and dedicated.
2. One row per order_id survives, keeping an arbitrary occurrence.
3. Delta adds ACID transactions, time travel, and schema enforcement.
4. When set logic is clearer in SQL but complex transforms need Python.
5. Full scan plus skewed inference; prefer explicit schemas at scale.

Week 4:

1. Parquet data files plus the JSON transaction log.
2. Rewrites Parquet with sorting/compression optimized for Fabric engines.
3. It compacts many small files into fewer, larger ones.
4. Vacuuming below retention deletes old versions, destroying recovery history.
5. `RESTORE TABLE ... TO VERSION AS OF n.`

Week 5:

1. When you need direct T-SQL bulk loads from OneLake/ADLS with transactional control.
2. Scale or pause compute independently while data persists in OneLake.
3. A Fabric Warehouse with T-SQL pipelines; stored procedures port most directly.
4. The Capacity Unit (CU), sized by F SKU.
5. Pay-as-you-go billing stops while paused, cutting idle spend.

Week 6:

1. It is generated from the Lakehouse; writes go through Spark or pipelines.
2. A view (or table-valued function) encapsulating the join logic.
3. RLS filters rows per user; object-level security grants or denies whole objects.
4. It reads the external data in place through the shortcut.
5. Full T-SQL tooling, source control, and existing DBA workflows.

Week 7:

1. The grain defines what one row means; mismatched columns corrupt aggregates.
2. Transaction, periodic snapshot, accumulating snapshot.
3. Standardized dimensions joinable consistently across multiple fact tables.
4. When history of attribute changes must be preserved for analysis.
5. Setting `is_current = false` and `valid_to` on the matched row.

Week 8:

1. Guardrail limits exceeded, or unsupported query shapes and RLS paths.
2. DirectLake reads Parquet directly; V-Order maximizes compression and pruning.
3. Framing loads recent Delta data into memory for fast queries.
4. Fallback events in the Capacity Metrics app and model telemetry.
5. Small models needing guaranteed cached performance or full feature parity.

Week 9:

1. Table lists and rules live in configuration, not hardcoded per pipeline.
2. The ForEach activity.
3. Breaking changes remove/rename columns; non-breaking changes only add columns.
4. The producer workspace owns and versions it; consumers validate against it.
5. Appending new columns automatically; default writes reject schema mismatches.

Week 10:

1. Where cleansed rows land: Lakehouse, Warehouse, Azure SQL, and similar.
2. Staging isolates transformation errors before destination writes.
3. Reloading full history; only new or changed slices are refreshed.
4. A few bad rows should not break the SLA or corrupt Gold aggregates.
5. Complex logic, large scale, or reuse beyond Power Query's strengths.

Week 11:

1. When sources live inside an Azure VNet without public endpoints.
2. A single gateway node is a single point of failure.
3. Managed/service identities or vaulted secrets; never hardcoded keys.
4. It makes outbound-only connections; no inbound public IP is exposed.
5. In-flight activities fail; reruns must be idempotent by design.

Week 12:

1. On Failure runs only on error; On Completion runs either way.

2. Status, duration, and per-activity run history across items.
3. It makes alerts actionable: rerun steps, backfill, escalation owner.
4. Watermark/manifest-driven loads so reprocessing never duplicates rows.
5. Compensating actions (rollback, quarantine), not just notifications.

Week 13:

1. Event Hubs and custom apps; destinations include KQL databases.
2. It enables downstream deduplication under at-least-once delivery.
3. How far back events can be replayed after a downstream failure.
4. KQL for real-time analytics; Lakehouse for durable batch history.
5. Duplicates must be expected and merged on event_id.

Week 14:

1. It stores only the aggregated delta per new ingested batch instead of rescanning raw rows; `arg_max` collapses replayed copies of the same `EventId`, keeping the latest arrival by `ingestion_time()`.
2. State and scan cost grow without bound: every query/materialization cycle re-reads more history, raising memory, CPU, and latency until the Eventhouse falls behind the ingest rate.
3. Join on the shared key, then filter on `datetime_diff` between the two event times within the window; late arrivals fall outside the window and are dropped or mis-matched, skewing the joined aggregates.
4. Generated at the producer before the first send and enforced at the first queryable boundary (KQL materialized view or Delta MERGE on `event_id`), so replay never duplicates rows downstream.
5. It records the last processed event time/offset so a restarted consumer resumes where it stopped; without it, restarts replay events and re-emit duplicates.

Week 15:

1. Data source (trigger), condition, and action.
2. It suppresses noise from single transient spikes.
3. Teams messages, emails, and Power Automate/custom workflows.
4. Via "Set alert" on the visual, creating a Reflex item.
5. Near-real-time: seconds to minutes after the condition is met.

Week 16:

1. Dashboards for shareable tiled visuals; querysets for ad-hoc exploration.
2. Repeated queries hit the source; load grows with refresh frequency.
3. Syncing curated warehouse data back into operational CRM/support tools.
4. A stable business key, e.g., the Dataverse record ID.
5. Freshness vs. CU cost and API throttling; nightly is cheaper but staler.

Week 17:

1. Executable PySpark code.
2. Speed and cost; validate assumptions before full-cluster scans.
3. Future-information leakage and training/serving skew.

4. Delta tables are governed, time-travelable, and shareable.
5. Distributed training/scoring on Spark with no data export.

Week 18:

1. Parameters, metrics, artifacts, and the model itself.
2. Versioning, aliasing, and governed promotion.
3. Dataset version, code commit, and environment.
4. Compare runs by metric and parameters in the experiment UI/MLflow.
5. Instant rollback to a previous registered version.

Week 19:

1. Distributed deep learning and prebuilt Azure AI service connectors.
2. Keep secrets in Key Vault or environment variables, retrieved at runtime.
3. Batching amortizes latency and cost across many rows.
4. Code, queries, and narrative insights.
5. A Gold Lakehouse table feeding Power BI.

Week 20:

1. Applies a registered MLflow model as a transformation over a DataFrame.
2. Scheduled bulk scoring beats per-request latency and cost here.
3. Feature distributions diverging from training baselines.
4. Pipelines give reproducible, monitored, retryable runs with alerts.
5. The downstream contract: stable output columns for Power BI and consumers.

Week 21:

1. Contributors write items; Viewers only read shared content.
2. It limits blast radius; workspace roles grant everything inside.
3. Folder/table-level data access within OneLake.
4. A filter predicate (region = 'US') applied per role at query time.
5. Accumulated over-broad grants; review workspace/item permissions quarterly.

Week 22:

1. Downstream items inherit the label and its protection.
2. Promoted is self-endorsed; Certified is centrally approved.
3. It shows upstream/downstream impact before changing or deleting items.
4. Tenant-wide adoption, sensitivity coverage, and audit insights.
5. Domain autonomy with centralized Purview governance.

Week 23:

1. Sustained CU overage beyond the smoothing windows.
2. Items redeploy from the repository into any workspace.
3. OneLake data is not automatically replicated across regions.
4. Background jobs starving interactive capacity.

5. It cuts idle CU spend in pay-as-you-go dev/test capacities.

Week 24:

1. Data engineering and analytics-solution implementation domains.
2. Large Delta tables where import refresh is too slow.
3. Re-study the lowest-scoring domains first, then re-mock.
4. DirectQuery fallback events in the Capacity Metrics app.
5. Capstone average at least 80% and mock-exam average at least 75%.

11 Appendix E: Datasets & Project Data

- **NYC Taxi (TLC trip records)** — available as a built-in Fabric sample dataset and published as public Parquet (<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>, mirrored on Azure Open Datasets). Use for Milestones 1–2: medallion Lakehouse, warehouse star schema, DirectLake reports. Multiple years exceed 100M rows; 1–5M rows suffice for the capstones.
- **Olist Brazilian e-commerce** (<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>) — roughly 100k orders across 9 related CSVs (orders, items, customers, products, payments, reviews). Use for Milestones 3 and 5: metadata-driven ingestion and churn/feature-engineering capstones.
- **Streaming events** — generate credit-card swipes and IoT telemetry synthetically for Milestone 4 (and PII data for Milestone 6) with the Python `faker` library; target 5–20k events per minute during demos.
- **Synthetic data note:** use `faker` for CRM/PII-style tables, seeded for reproducibility. Volume guidance: Milestone 1 $\approx$ 1–10M CSV rows; Milestone 2 $\approx$ 5M fact rows; Milestone 3 $\approx$ 9 Olist tables re-ingested daily; Milestone 5 $\approx$ 100k scoring rows. Commit generator scripts, not generated data, to your repositories.

12 Appendix F: Portfolio & Presentation Guide

GitHub README structure (every capstone):

1. Problem statement (2–3 sentences, business framing).
2. Architecture diagram first — reuse the TikZ figure from this syllabus or draw your own.
3. Setup instructions that run from a clean environment.
4. Screenshots: pipeline run, dashboard, alert, Monitoring Hub.
5. Capacity cost estimate (CU consumption) and the optimizations applied.
6. Lessons learned: what broke and what you would do differently.

Resume bullet templates (two per milestone):

- **M1:** Built a Spark medallion lakehouse on Microsoft Fabric (Bronze/Silver/Gold) with Delta time-travel recovery, processing N rows/day.
- **M1:** Virtualized multi-cloud data with OneLake shortcuts, eliminating duplicate storage across ADLS and S3.

- **M2:** Delivered a DirectLake Power BI star schema over N rows with sub-second response and row-level security.
- **M2:** Implemented Kimball dimensional models (conformed dimensions, SCD Type 2 MERGE) in PySpark on a Fabric Lakehouse.
- **M3:** Built a metadata-driven ingestion framework onboarding N tables with zero new pipelines.
- **M3:** Added data-contract validation and quarantine patterns, eliminating batch failures from bad rows.
- **M4:** Engineered a real-time fraud pipeline (Eventstream → KQL → Data Activator) alerting within N seconds.
- **M4:** Implemented reverse-ETL syncs from Gold tables to Dataverse/CRM with idempotent, key-based upserts.
- **M5:** Operationalized an MLflow-tracked churn model with daily batch scoring over N rows via Fabric pipelines.
- **M5:** Integrated Azure OpenAI sentiment enrichment into a Spark pipeline processing N reviews.
- **M6:** Designed Git-integrated CI/CD (Dev → Test → Prod) with Purview labels and least-privilege access.
- **M6:** Authored the DR plan (Git recovery + Delta time travel + dual-workspace BCDR) and passed DP-600.

Talking about your project in interviews: lead with the business problem and one number (rows, latency, or cost saved), walk the architecture diagram top to bottom, then describe one failure you hit and how you fixed it. Interviewers probe for trade-offs — be ready to explain why you chose Lakehouse vs. Warehouse, Import vs. DirectLake, or batch vs. streaming, and what you would change at 10× scale.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone	Status
1	OneLake & Fabric Foundations	OneLake Shortcuts	☐
2	The Medallion Architecture	Bronze/Silver/Gold Lakehouses	☐
3	Apache Spark in Fabric	PySpark Data Cleansing	☐
4	Delta Lake Optimization	Cap 1: Spark Medallion Lakehouse	☐
5	Synapse Data Warehouse	COPY INTO from OneLake	☐
6	SQL Analytics Endpoint	Cross-Lakehouse T-SQL Views	☐
7	Data Modeling for Power BI	Fabric Semantic Models & DAX	☐
8	Power BI DirectLake Mode	Cap 2: End-to-End Analytics Workflow	☐
9	Data Factory Pipelines	Dynamic Pipeline with ForEach Loop	☐
10	Dataflows Gen2	Power Query Online to Lakehouse	☐
11	Connectivity & Hybrid	On-Premises Data Gateway config	☐
12	Orchestration	Cap 3: Automated Batch ELT	☐
13	Eventstreams	Custom App Eventstream Ingestion	☐
14	KQL Databases	KQL Queries & Anomaly Detection	☐
15	Data Activator (Reflex)	Automated Alerts via Teams/Email	☐
16	Real-Time Dashboards	Cap 4: Real-Time Fraud Telemetry	☐
17	Data Science Foundations	Data Wrangler code generation	☐
18	Model Training & MLflow	MLflow Autologging & Registration	☐
19	SynapseML & AI Integration	Azure OpenAI sentiment analysis	☐
20	MLOps & Batch Scoring	Cap 5: End-to-End MLOps Pipeline	☐
21	Security & Access Control	SQL RLS and Workspace Roles	☐
22	Governance, Purview & GDPR Erasure	Sensitivity Labels, Lineage & Crypto-Shredding	☐
23	Administration & CI/CD	Git Integration & Deployment Pipelines	☐
24	Exam Prep (DP-600)	Cap 6: Secure Governed Platform	☐

"Consistency is the architecture of mastery."

Best of luck on your 6-month journey to becoming a Master Microsoft Fabric Analytics Engineer!